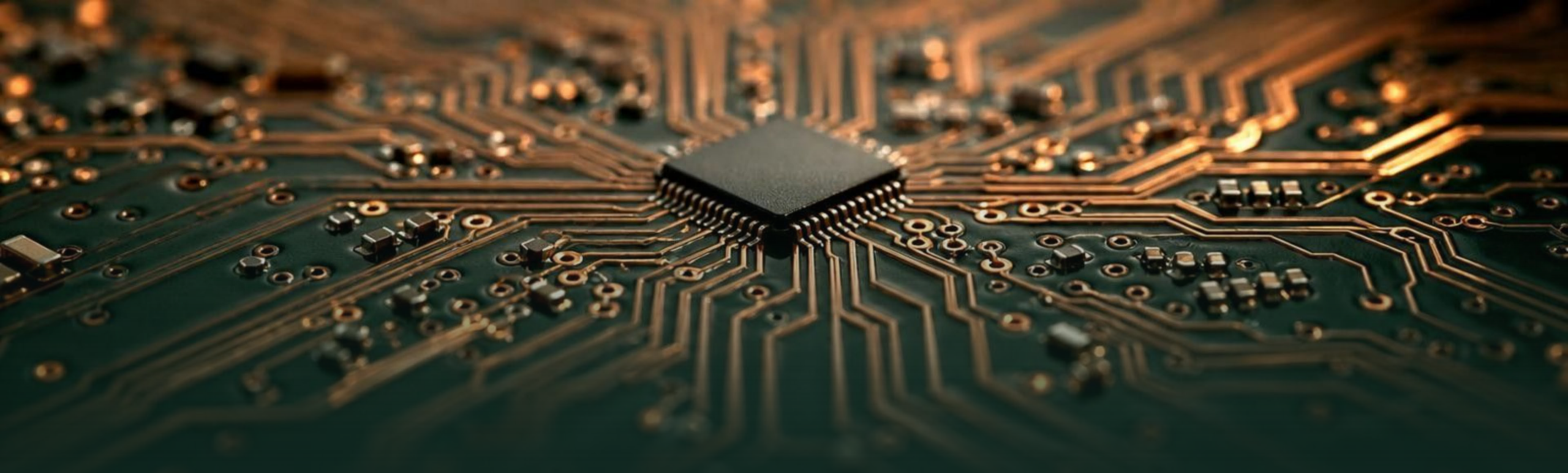


# Project Q

## Verilog in. A quantum -ready puzzle and an honest scorecard out.

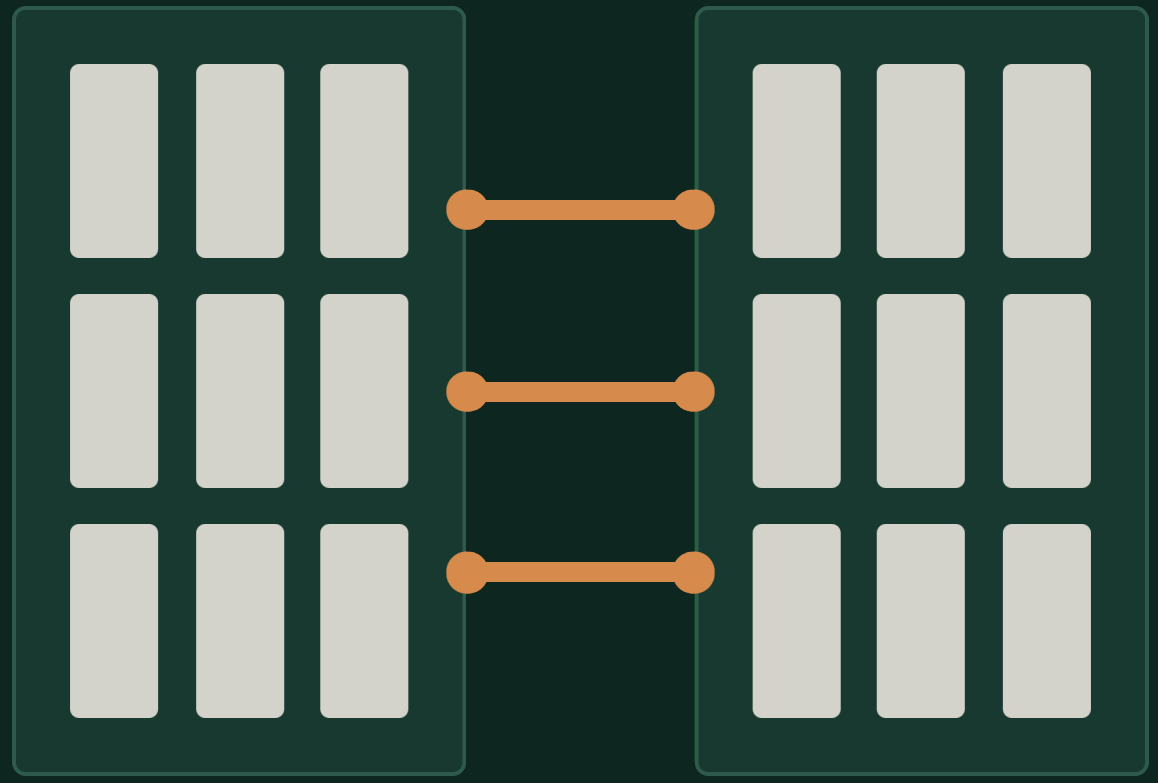
Team: Dev Arsh Singh · Shrugal Agarwal · Moath Saleh · Prisha Lala

IBM Qiskit Fall Fest 2026

Speaker: Prisha Lala

# Big chips don't fit on one board

- Before a chip is manufactured, it is tested on FPGAs: reprogrammable chips that can imitate any circuit
- A large design (a full CPU) is too big for one FPGA, so it is split across several boards
- Modern processors do the same with chiplets: several small dies in one package
- Every wire that crosses between boards or dies costs a scarce pin, speed and power



Speaker: Moath Saleh

# The puzzle: cut it in two

**Goal:** split the gates into two halves with

- the **fewest wires crossing** (the "cut"), and
- **both halves about the same size** (the "balance").

The two goals fight: put everything on one chip and the cut is 0, but the split is useless.

Why it's hard: every gate goes left or right, so n gates give $2^n$ possible splits.

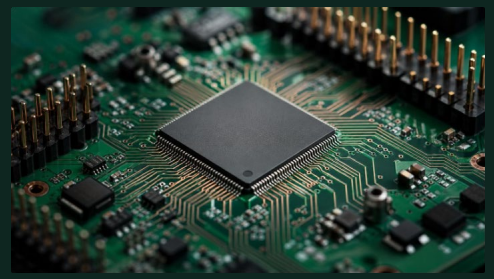
| Gates | Possible splits |
|---|---|
| 6 (our test circuit, c17) | 64 |
| 20 | about 1 million |
| 100 | about $10^{30}$ |

Speaker: Moath Saleh

# Our bridge: one command

Command `project-q partition design.v --solver qaoa`

- **Who it's for:** hardware engineers who write Verilog but have never touched quantum computing
- **What it does:** turns their circuit into a problem a quantum computer can attempt, solves it with QAOA in Qiskit, and scores the answer against the best classical tools
- **What we claim:** the bridge and the honest benchmark. Not that quantum wins today.



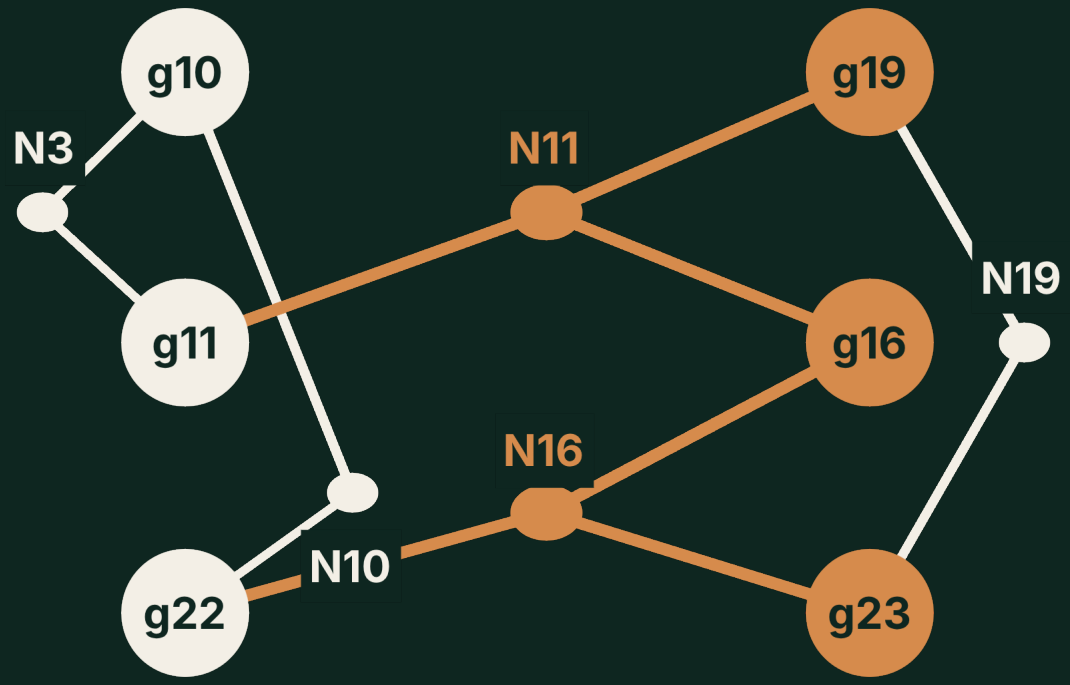
| The challenge asks for | Project Q |
|---|---|
| A real problem outside quantum | Splitting chip designs across FPGAs and chiplets |
| A useful bridge | Verilog in, quantum-ready problem and scorecard out |
| A quantum idea in Qiskit | QAOA solving the hardest core of the problem |

Speaker: Moath Saleh

# Wires are group chats, not phone calls

- In a normal graph, an edge joins exactly 2 things, like a phone call
- In a chip, one wire can feed many gates, like a group chat. A graph whose edges can join more than 2 things is a **hypergraph**
- A wire is **cut** if its gates sit on different chips. It counts once, however many gates are on each side

Our test circuit c17: 6 NAND gates, 5 wires. The best balanced split cuts just **2** wires, and only one split (and its mirror image) reaches it.




Speaker: Shrugal Agarwal

# One score formula: lower is better

Give each gate a bit: **$x = 0 \rightarrow$ chip A, $x = 1 \rightarrow$ chip B.**

- A 2-gate wire scores $x_1 + x_2 - 2x_1x_2$ : 0 if both on the same side, 1 if split
- 3-gate wires have an exact formula too; 4–8 gate wires are approximated; wires with more than 8 gates are left out of the formula but still counted in the true cut; huge wires (clock, reset; more than 50 gates) are dropped when the netlist is read
- Balance penalty: $\lambda\,(\Sigma\, x_i - n/2)^2$ , zero for a perfect 50/50 split

This shape of formula is a **QUBO** (Quadratic Unconstrained Binary Optimization), the format quantum optimizers accept.

"Why $\lambda = 2.0$? It is the default we chose from the energy margin: a lopsided split costs more than the best answer. Our $\lambda$ sweep does not show that $\lambda = 2$ samples better, so we report the sweep (slide 13)."

Speaker: Shrugal Agarwal

# From a score to quantum energy

$$x_1 + x_2 - 2x_1x_2 \rightarrow (1 - z_1z_2)/2$$

Physics reads a qubit as z = +1 or −1. Swap **x = (1 − z) / 2**

The wire cost becomes **(1 − z₁z₂) / 2**agree → 0, disagree → 1

The whole formula becomes an energy (a Hamiltonian). **Lowest energy = best split**

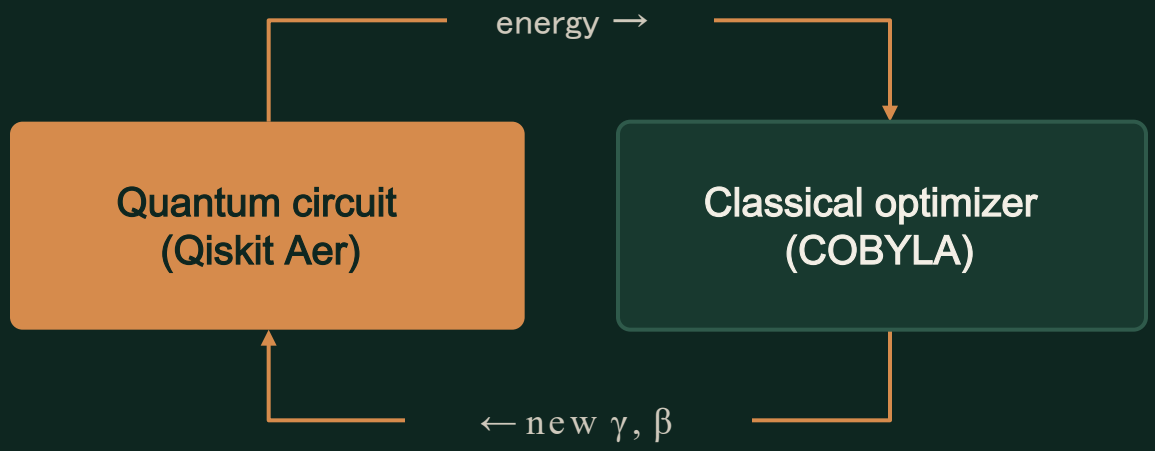
The $z_1z_2$ term is exactly what the quantum gate RZZ implements

Speaker: Shrugal Agarwal

# QAOA in one picture

A quantum circuit with two adjustable knobs, tuned by ordinary software.

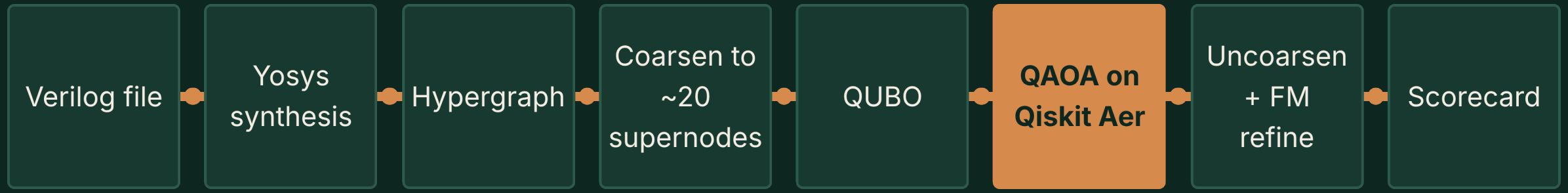
1. **Start fair:** every possible split equally likely
2. **Cost layer (knob γ):** tag each split with a phase based on its score
3. **Mixer layer (knob β):** let the tags interfere, so low-score splits become more likely
4. **COBYLA tunes γ and β** on the simulator's energy, with several restarts.
5. **Measure 4,000 shots** at the best γ, β. Keep the best balanced split, re-checked by plain Python.




"A quantum computer does not try every answer at once. It shifts the odds toward good ones."

Speaker: Shrugal Agarwal

# What we are building: eight steps, only one quantum.




Python package with a 251 - test suite; every phase has a pass/fail gate

Runs without Yosys installed: synthesized netlists are committed in the repo

Stack: Qiskit 2.6, Qiskit Aer, IBM Runtime, Yosys, NumPy, NetworkX

Speaker: Dev Arsh Singh

# How we know the formula is right

64 / 64

Gate 1, on c17:

- Checked all 64 possible splits: the formula's cut equals the true cut on every one
- The lowest score is exactly the best balanced split (cut 2)
- Energy gap to the next -best answer: 1.0 (it must be above 0)
- The "true cut" checker is separate plain Python that shares no code with the formula, so a bug in the quantum side cannot hide

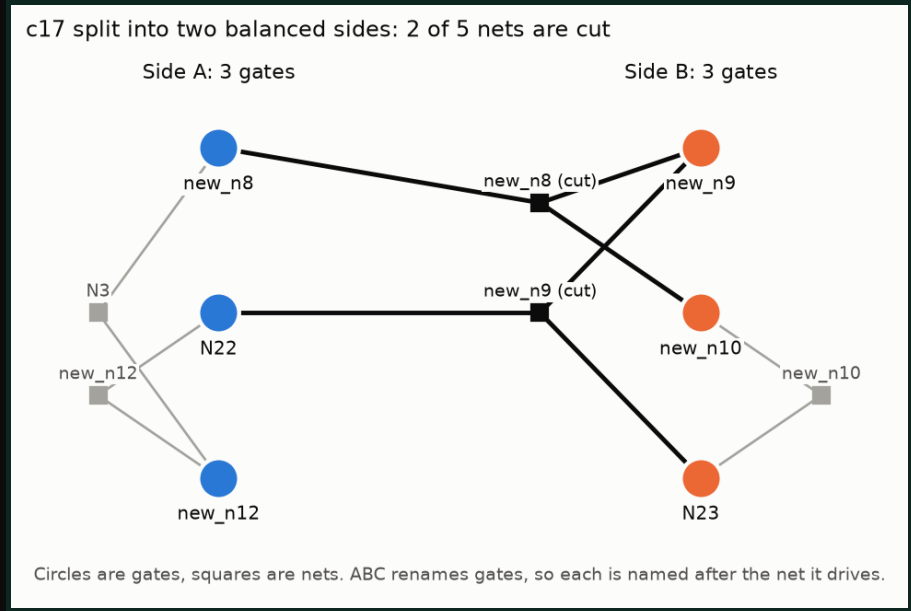
Speaker: Dev Arsh Singh

# Live demo: notebooks/project_q_executed.ipynb (runs top to bottom without Yosys)

Command

```
project-q partition benchmarks/c17.v --solver qaoa
```

```
c17: 6 nodes, 5 nets, solver qaoa
cut 2  best known 2 (brute_force)  gap 0.0%
balanced True  balance ratio 1.000  runtime 3.48 s
balanced shots 68.7%  optimal shots 10.2%
side A: new_n9, new_n10, N23
side B: new_n8, new_n12, N22
saved results/c17_qaoa_20261106T101644555087.json
```




Speaker: Dev Arsh Singh

# The honest scorecard

Same problem, same ruler, every solver.

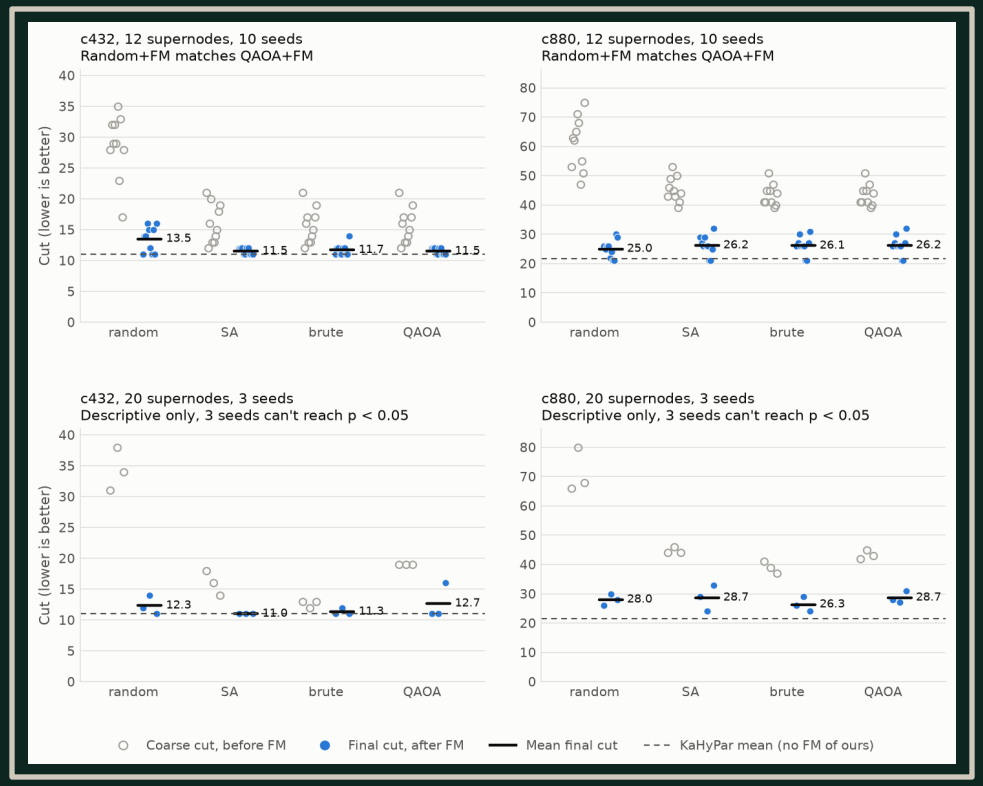
| Solver | What it is | c17 cut | c432 cut | c880 cut |
|---|---|---|---|---|
| Brute force | tries every split (exact, up to 24 gates) | 2 | — | — |
| Random | random balanced split | 3.6 | 113.2 | 299.4 |
| Simulated annealing | classical trial and error | 2.0 | 13.8 | 25.7 |
| Kernighan–Lin | textbook gate-swapping | 2.0 | 19.1 | 41.4 |
| KaHyPar | state-of-the-art hypergraph partitioner | 2.0 | 11.0 | 21.6 |
| **QAOA (ours)** | **quantum, simulated, whole circuit** | **2.0** | **n/a (over 24 qubits)** | **n/a (over 24 qubits)** |
| **QAOA + coarsening + FM (ours)** | **quantum on a 20-supernode version, then refined** | **—** | **12.7** | **27.7** |

Cuts are means over 5 seeds (3 for the 20-qubit QAOA row); lower is better. Brute force is exact only on c17. On c432 and c880 the best cut is the best we found, not proven optimal. QAOA does not beat the best classical solver here, and we report that. KaHyPar is optional; without it a labelled Kernighan–Lin fallback runs.

Speaker: Prisha Lala

# Is QAOA doing anything?

**Ablation test:** in the big-circuit pipeline, swap QAOA for random and for simulated annealing at the coarsest level, keep everything else the same.

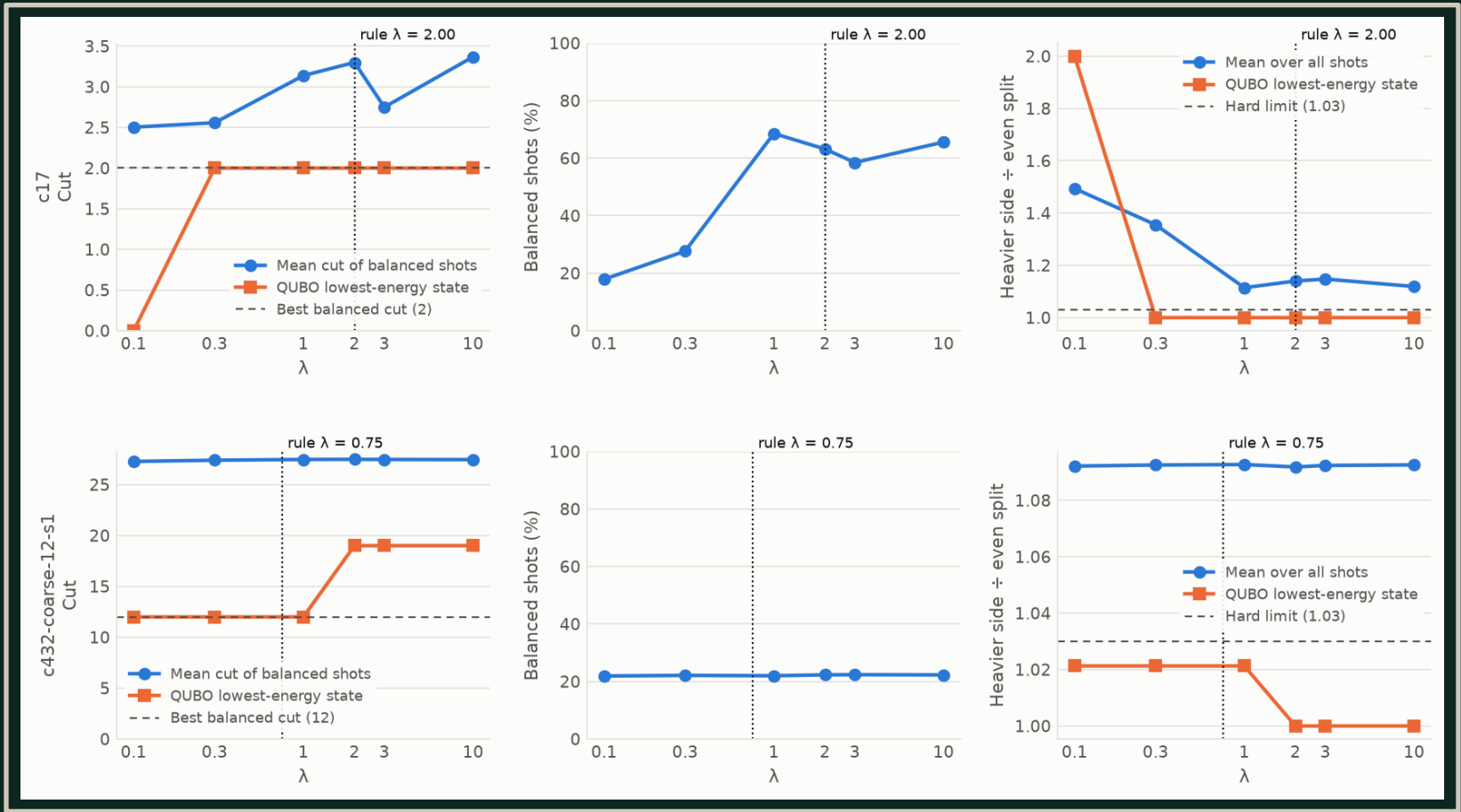



Ablation on c880 (20 supernodes, 3 seeds, descriptive only): random + FM 28.0, simulated annealing + FM 28.7, QAOA + FM 27.7. QAOA was lower on 1 seed, tied on 1, higher on 1 (sign test p = 1.000), so we cannot say QAOA is what helps.

**"If random + refinement matches QAOA + refinement, we say so."**

Speaker: Prisha Lala

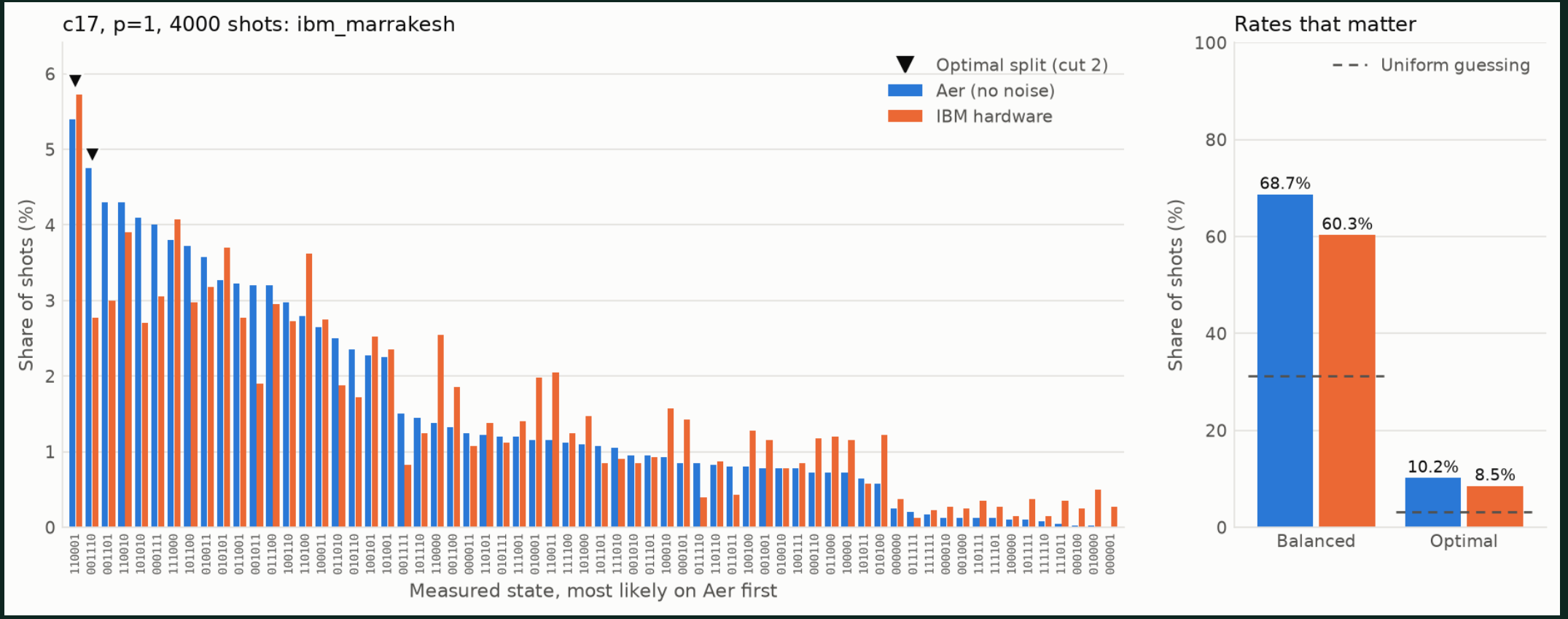
# The λ sweep does not show λ = 2 is optimal




λ sweep on c17: λ = 1 gave more balanced shots than λ = 2, so λ = 2 is a default, not a proven optimum.

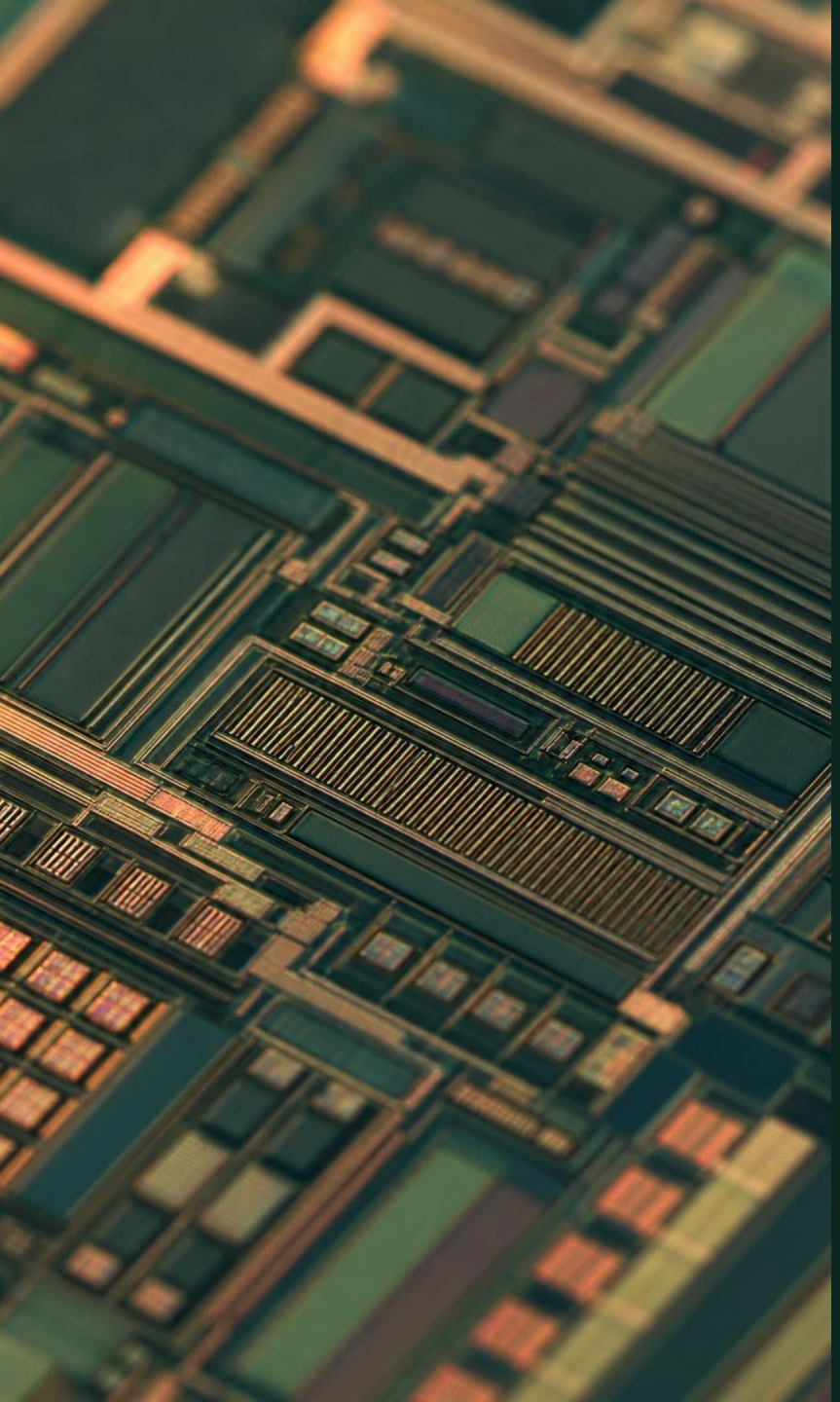
Speaker: Prisha Lala

# One real IBM hardware run




One real IBM run (ibm_marrakesh, c17, 6 qubits, 4,000 shots). Balanced shots: hardware 60.3%, simulator 68.7%, random guessin g 31.3%. Optimal shots: 8.5%, 10.2%, 3.1%. Noise cost balance but it stayed far above guessing. One run, so no trend is claimed , and no quantum advantage.

Speaker: Prisha Lala



# Limits, and what comes next

## Where it falls short today

- Simulators stop near 25 qubits: QAOA is capped at 24 and big circuits shrink first
- Wires touching 4+ gates are approximated
- Classical tools win at this size, and we show by how much
- One real hardware run (6 qubits): it says nothing about larger circuits or beating classical solvers

## Next

- Wire routing on a grid, a simplified 2-pin version of the VLSI routing class in IBM's QOBLIB benchmark library
- Splitting into more than two chips
- Re-running the same scorecard as quantum hardware improves

Speaker: Prisha Lala

# Write Verilog. Get a quantum -ready problem and an honest scorecard.

Hardware engineers can now ask quantum questions without a physicist.

Built with AI assistance (Claude Code). Every team member can explain their part.

**Dev Arsh Singh:**
Phases 0–2: Yosys parser, hypergraph, QUBO, classical baselines; project setup, review and final audit

**Shrugal Agarwal:**
Phases 3–4: QAOA solver, command -line tool, scorecard, KaHyPar baseline; the formulation and QAOA reasoning

**Moath Saleh:**
Phases 6 and 9: experiments, ablation, λ sweep, plots, README, executed notebook; problem motivation

**Prisha Lala:**
Phases 5 and 8: coarsening and FM refinement, IBM hardware script; results and scorecard

Repo: github.com/DevArshSingh/Project -Q

Speaker: Prisha Lala